



GPU Portfolio Optimization

CS4220 – Final Project

Ethan Ngo

California State Polytechnic University, Pomona

Abstract

This project accelerates Markowitz mean-variance portfolio optimization using GPU computing. Given a universe of N assets and T days of historical returns, the classical formulation requires constructing an $N \times N$ covariance matrix and solving a regularized optimization problem via gradient descent, both of which scale poorly on a CPU. We implement a custom CUDA kernel for parallel covariance computation and use NVIDIA cuBLAS for the matrix-vector multiplications at the core of each gradient step. Benchmarked on an NVIDIA GPU (NCSA Delta cluster) against a C++17 CPU baseline, our GPU pipeline achieves up to $125 \times$ speedup on covariance computation across portfolio sizes of $N = 50$ to 400 assets, with 5 years of daily return data ($T \approx 1260$ days). Total end-to-end speedup reaches approximately $2.5 \times$, with the gradient descent loop remaining the primary bottleneck due to the relatively small matrix sizes tested.

Introduction

Portfolio optimization is a foundational problem in computational finance. The Markowitz mean-variance framework seeks to find asset weights w that maximize expected return $\mu^T w$ while minimizing portfolio variance $w^T \Sigma w$, subject to the constraint that weights sum to 1. Formally, the objective is to minimize:

$$f(w) = \lambda \cdot w^T \Sigma w - \mu^T w$$

where Σ is the $N \times N$ covariance matrix of asset returns and μ is the vector of mean returns.

As the number of assets N grows from tens to hundreds or thousands, computational cost increases significantly. Covariance matrix construction requires $O(N^2 \cdot T)$ operations, and each gradient descent iteration requires a matrix-vector multiply Σw of cost $O(N^2)$. With thousands of iterations and large portfolios, runtime on a CPU becomes a practical bottleneck for real-time or high-frequency applications.

GPUs are well-suited to this problem: covariance computation is embarrassingly parallel (each cell Σ_{ij} is independent), and matrix-vector operations are a natural fit for hardware-optimized BLAS routines. This project implements and benchmarks a full GPU-accelerated optimization pipeline against a CPU baseline, measuring speedup across a range of portfolio sizes.

Methodology

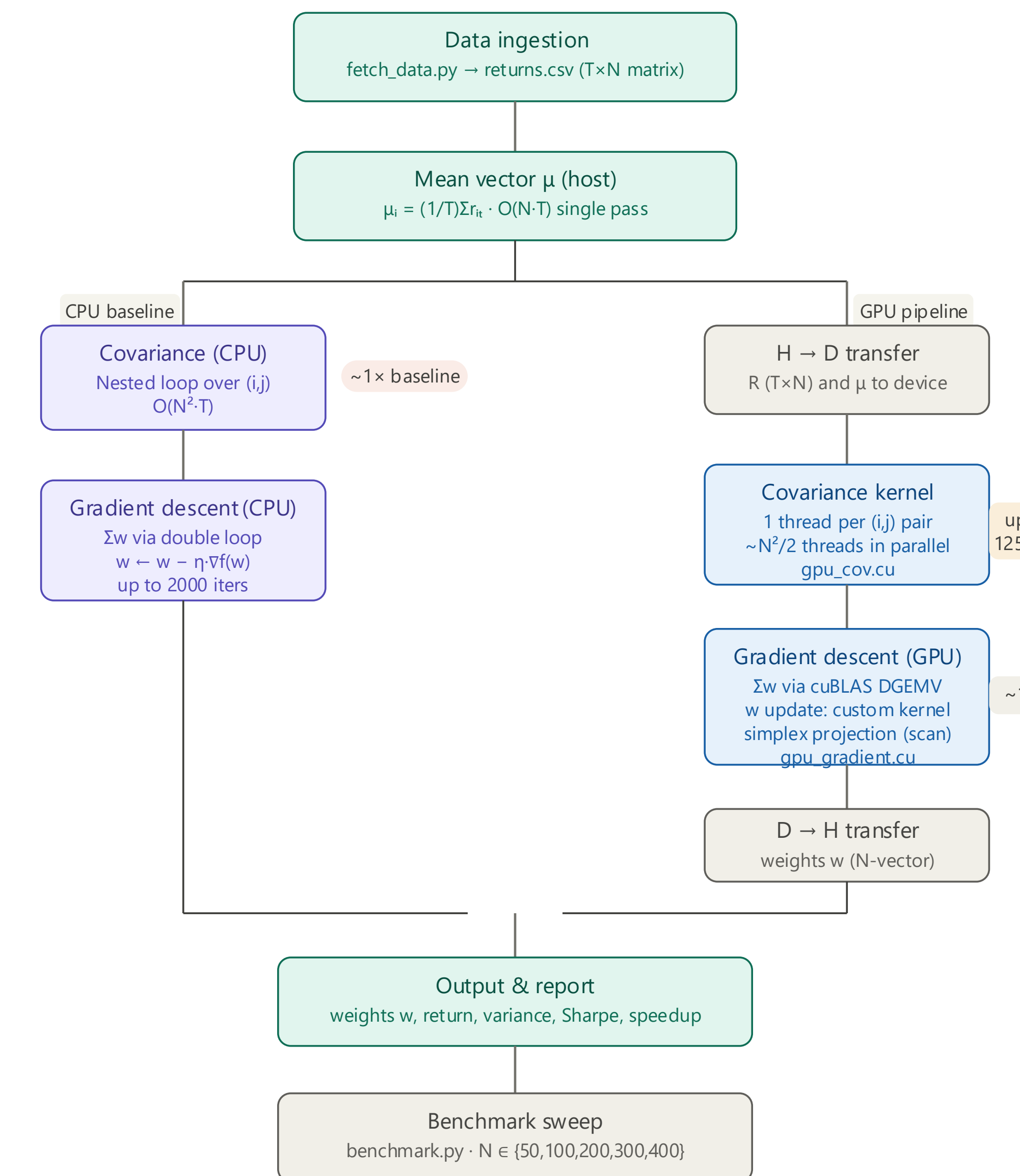


Figure 1. Data Pipeline

CPU Baseline The CPU pipeline is implemented in C++17. It loads historical daily returns from CSV, computes the per-asset mean return vector μ , constructs the full $N \times N$ covariance matrix via a nested loop ($O(N^2 \cdot T)$), and solves the optimization using projected gradient descent, iteratively updating weights as $w \leftarrow w - \eta \cdot \nabla f(w)$, where the gradient is

$$\nabla f(w) = 2\lambda \Sigma w - \mu.$$

After each step, weights are projected onto the simplex (sum-to-one constraint). The baseline runs entirely on a single CPU core.

GPU Pipeline The GPU pipeline mirrors the CPU baseline but offloads the two most expensive operations:
Covariance Kernel (gpu_cov.cu): A custom CUDA kernel computes the covariance matrix in parallel. Each thread is assigned one (i, j) pair and independently computes $\Sigma_{ij} = (1/(T-1)) \cdot \sum_t (r_{it} - \mu_i)(r_{jt} - \mu_j)$. The kernel uses coalesced memory access patterns over the T -length return vectors for each asset pair.

Gradient Descent (gpu_gradient.cu): The core matrix-vector product Σw is computed via cuBLAS DGEMV. A custom CUDA kernel then applies the parallel weight update $w_i \leftarrow w_i - \eta \cdot \nabla_i$ across all N weights simultaneously, followed by a simplex projection step.

Benchmark Setup Both pipelines are timed using CUDA events (GPU) and wall-clock timers (CPU) across portfolio sizes $N \in \{50, 100, 200, 300, 400\}$ with $T \approx 1260$ trading days. Data is fetched from Yahoo Finance via `yfinance`.

Results

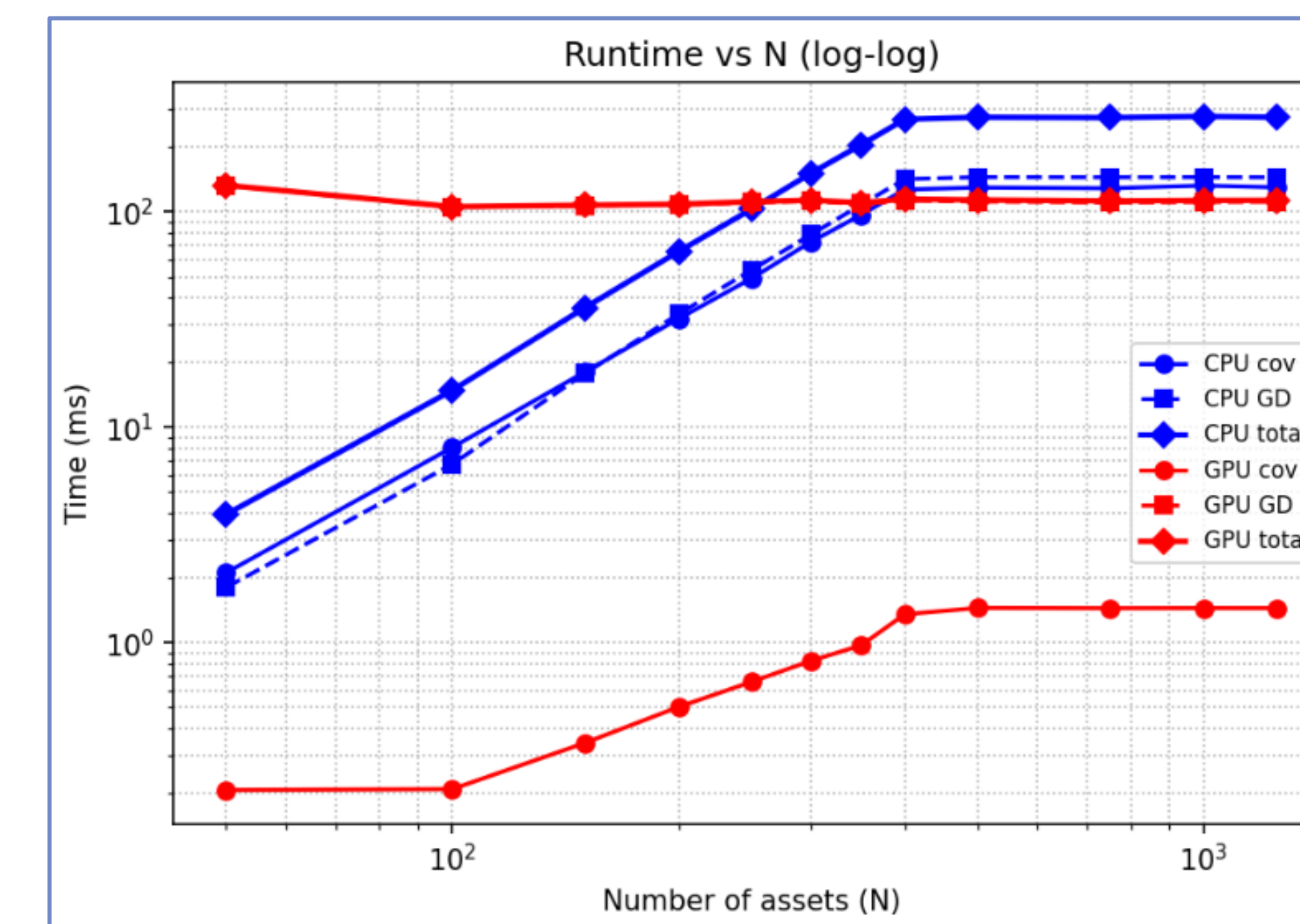


Figure 2. CPU vs GPU runtime scaling across portfolio sizes

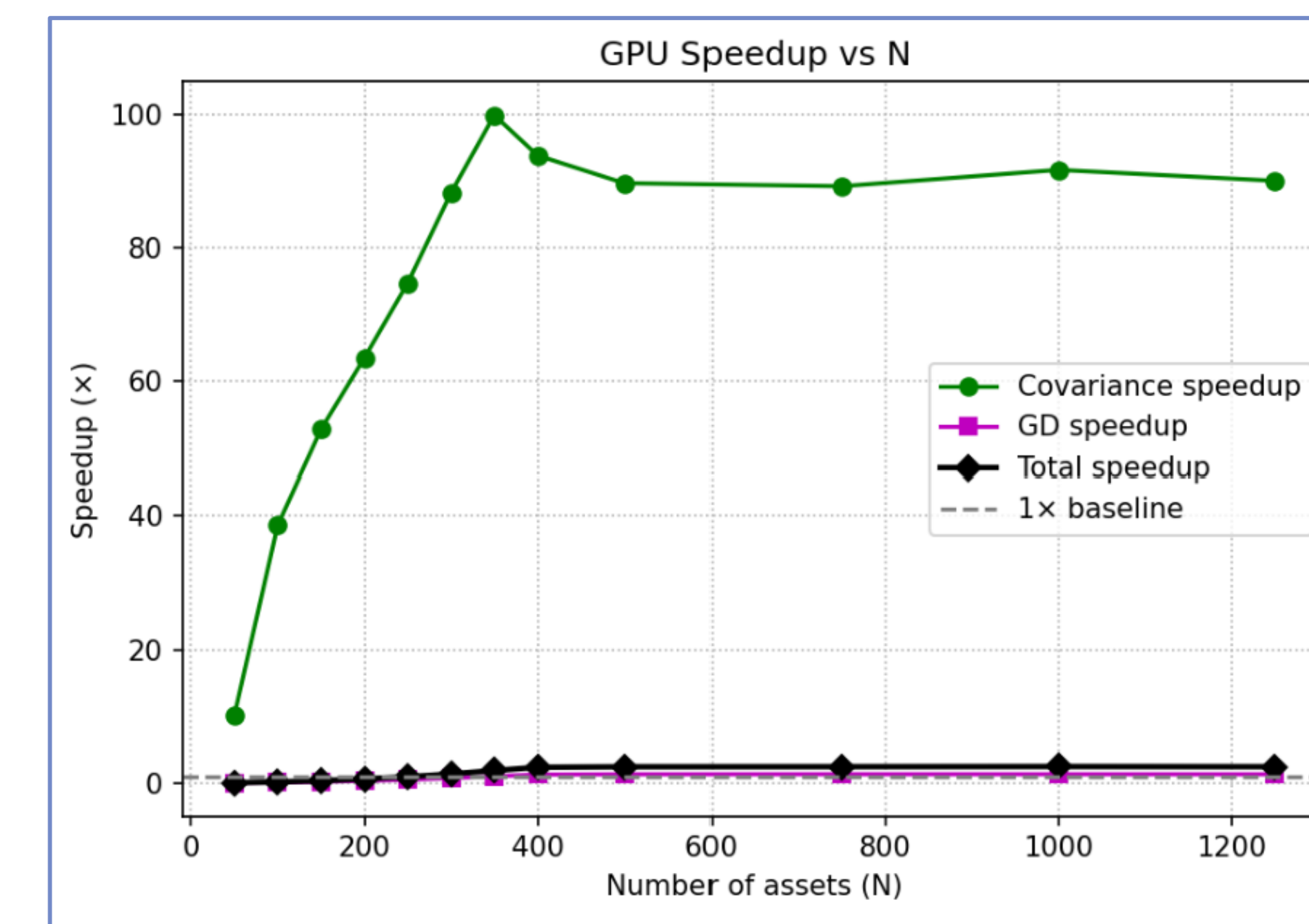


Figure 3. GPU speedup vs portfolio size

Figure 2 shows runtime on a log-log scale from $N = 50$ to $N = 1250$. The CPU covariance line climbs steeply with a clear $O(N^2)$ slope, growing from ~ 2 ms at $N = 50$ to over 200 ms at $N = 1250$, while the GPU covariance line stays nearly flat below 2 ms throughout. CPU and GPU total lines converge and overlap at larger N , confirming that gradient descent dominates end-to-end wall time on both architectures.

Figure 3 quantifies the speedup directly. Covariance speedup rises steeply from $10 \times$ at $N = 50$, peaks at $100 \times$ at $N = 350$, then plateaus at a consistent $90 \times$ through $N = 1250$. The early rise reflects $O(N^2 \cdot T)$ scaling, parallel work grows quadratically while GPU overhead stays fixed. The plateau indicates the kernel has become memory-bandwidth bound as the A40's memory bus saturates at larger N . Gradient descent and total speedup remain flat near $1-3 \times$ across the entire range, despite the $90-100 \times$ covariance advantage, gradient descent running 2000 cuBLAS DGEMV iterations dominates wall time and resists acceleration, a direct illustration of Amdahl's Law.

Conclusion

This project demonstrates that GPU acceleration delivers transformative speedup for the covariance computation step of mean-variance portfolio optimization, sustaining $\sim 90-100 \times$ speedup over a C++ CPU baseline across portfolio sizes up to $N = 1250$ assets. The custom CUDA covariance kernel scales efficiently with problem size, exploiting the $O(N^2 \cdot T)$ parallelism of the computation until the kernel becomes memory-bandwidth bound around $N = 400$, beyond which speedup plateaus at a consistent $90 \times$.

However, total end-to-end speedup reaches only $\sim 2-3 \times$ due to the gradient descent loop, which runs 2000 cuBLAS DGEMV iterations and dominates wall time regardless of portfolio size. This is a practical demonstration of Amdahl's Law, accelerating one component of a pipeline only improves total performance in proportion to that component's share of runtime.

The key takeaway is that GPU acceleration is most powerful when applied to the right bottleneck. Future work would target the gradient descent loop directly through a fully custom GD kernel, or replace iterative gradient descent with a direct solver such as a Cholesky decomposition, which would eliminate the iteration bottleneck entirely and allow the full pipeline to benefit from GPU parallelism.

Acknowledgements

Code & Reproducibility

Full source code, build instructions, and benchmark scripts are available at:

<https://github.com/ethan-ngo/FinalProject>